



Savitribai Phule Pune University

T. Y. B.Sc Cyber and Digital Science

Semester-VI

**CDS-368A Lab on
CDS-367A- Malware Analysis**

Professional Elective-IV

Lab Book

Name:_____

College Name:_____

Roll No.:_____ **Division:**_____

Academic Year : _____

CERTIFICATE

This is to certify that Mr./Ms. _____
Seat Number _____ of **T.Y.BSC (Cyber and Digital
Science) Sem-VI** has successfully completed Laboratory course
(Professional Elective-IV Lab on Malware Analysis) in the year ____.
He/ She have scored _____ mark out of 10 (For Lab Book).

Subject Teacher

H.O.D./Coordinator

Internal Examiner

External Examiner

Introduction

1. About the Workbook:

This workbook is intended to be used by T. Y. B.Sc (Cyber and Digital Science) Semester VI students for (Professional Elective-IV Lab on Malware Analysis) Assignments. This workbook is designed by considering all the practical concepts / topics mentioned in syllabus.

2. The objectives of this Workbook are:

- 1) Defining the scope of the course.
- 2) To bring the uniformity in the practical conduction and implementation in all colleges affiliated to SPPU.
- 3) To have continuous assessment of the course and students.
- 4) Providing ready reference for the students during practical implementation.
- 5) Provide more options to students so that they can have good practice before facing the examination.
- 6) Catering to the demand of slow and fast learners and accordingly providing the practice assignments to them.

3. How to use this Workbook:

The workbook contains Seven assignments

It is mandatory for students to complete all the assignments in given slot.

4. Instructions to the students:

Please read the following instructions carefully and follow them.

- 1) Students are expected to carry this workbook every time they come to the lab for practical.
 - 2) Students should prepare for the assignment by reading the relevant material which is mentioned in ready reference.
 - 3) Instructor will specify which problems to solve in the lab during the allotted slot and student should complete them and get verified by the instructor. However, student should spend additional hours in Lab and at home to cover all workbook assignments if needed.
- Students will be assessed for each assignment on a scale from 0 to 5

Not done	0
Incomplete	1
Late Complete	2
Needs improvement	3
Complete	4
Well Done	5

5. Instruction to the Instructors:

Make sure that students should follow above instructions.

- Explain the assignment and related concepts using white board if required or by demonstrating the software.
- Give specific input to fill the blanks in queries which can vary from student to student.
- Evaluate each assignment carried out by a student on a scale of 5 as specified above by ticking appropriate box.
- The value should also be entered on assignment completion page of the respective Lab course.

6. Instructions to the Lab administrator:

You have to ensure appropriate hardware and software is made available to each student.

Editors:

- 1) **Mr. Satyavan Kunjir**
- 2) **Mr. Sammed Bukshete**
- 3) **Mrs. Renuka Kulkarni**

Dr. D. Y. Patil ACS College, Pimpri, Pune
Dr. D. Y. Patil ACS College, Pimpri, Pune
Dr. D. Y. Patil ACS College, Pimpri, Pune

Reviewed By:

- 1) **Dr. Ranjit D. Patil**
- 2) **Dr. Sujata Patil**

Dr. D. Y. Patil ACS College, Pimpri, Pune
Dr. D. Y. Patil ACS College, Pimpri, Pune

Assignment Index

Sr. No.	Assignment Name.	Page Number
1	Setting up the Malware Lab	1
2	Static Analysis of Malwares	4
3	Dynamic Analysis of Malwares	14
4	Disassembly using IDA	19
5	Debugging malicious binaries	22
6	Introduction to Malware entry points and safeguarding	25
7	Toolkit for Malware Analysis	28
8	Introduction to network analysis	31
9	Research Paper Writing Activity	33

Signature of Instructor:

Assignment Completion Sheet

Sr. No.	Assignment Name	Marks (out of 5)	Teacher's Sign
1	Setting up the Malware Lab		
2	Static Analysis of Malwares		
3	Dynamic Analysis of Malwares		
4	Disassembly using IDA		
5	Debugging malicious binaries		
6	Introduction to Malware entry points and safeguarding		
7	Toolkit for Malware Analysis		
8	Introduction to network analysis		
9	Research Activity		
Total (Out of 40)			
Total (Out of 5)			

Signature of Instructor:

Assignment 1

Setting up the Malware Lab

Attempt Any One

Goals:

1. Install either VMware Workstation Player or Virtual Box on your system.
2. Install and configure a Linux-based operating system on a virtual machine.
3. Install and configure a Windows operating system on a virtual machine.
4. Install and configure a macOS operating system on a virtual machine.

Theory:

1. Downloading and Installing VMware/Virtual Box

What is Virtualization?

Virtualization is the process of creating virtual instances of computer hardware platforms, storage devices, and network resources. Tools like VMware and VirtualBox enable users to run multiple operating systems on a single physical machine.

Steps to Install VirtualBox:

1. Go to the official VirtualBox website: <https://www.virtualbox.org/>.
2. Download the installer for your operating system (Windows, macOS, or Linux).
3. Run the installer and follow the on-screen instructions.
4. Install the VirtualBox Extension Pack for additional features like USB support and RDP.

Steps to Install VMware Workstation Player:

1. Go to VMware's website: <https://www.vmware.com/>.
2. Download the free version of VMware Workstation Player.
3. Run the installer and follow the installation wizard.

2. Setting Up and Configuring a Linux VM

Steps:

1. **Download an ISO:** Choose a Linux distribution (e.g., Ubuntu, Fedora, or CentOS) and download its ISO file from the official website.
2. **Create a New VM:**
 - Open VirtualBox or VMware.
 - Click on "New" or "Create New Virtual Machine."
 - Set the VM name, type (Linux), and version (e.g., Ubuntu 64-bit).
3. **Allocate Resources:**
 - Assign memory (minimum 2 GB recommended).
 - Create a virtual hard disk (20 GB or more recommended).
4. **Attach ISO File:**
 - Go to the VM settings and add the ISO file as the bootable disk.
5. **Install Linux:**
 - Start the VM.

- Follow the installation prompts to install Linux.
- Set up a user account and configure system settings.

3. Setting Up and Configuring a Windows VM

Steps:

1. **Download an ISO:** Download a Windows ISO file from Microsoft's official website.
2. **Create a New VM:**
 - Open Virtual Box or VMware.
 - Click on "New" or "Create New Virtual Machine."
 - Set the VM name, type (Windows), and version (e.g., Windows 10).
3. **Allocate Resources:**
 - Assign memory (minimum 4 GB recommended for Windows 10).
 - Create a virtual hard disk (40 GB or more recommended).
4. **Attach ISO File:**
 - Go to the VM settings and add the ISO file as the bootable disk.
5. **Install Windows:**
 - Start the VM.
 - Follow the installation wizard to install Windows.
 - Activate the system using a valid product key.

4. Setting Up and Configuring a mac OS VM

Steps:

1. **Download mac OS:** Obtain a macOS installation file (ensure compliance with Apple's licensing terms).
2. **Create a New VM:**
 - Open Virtual Box or VMware.
 - Click on "New" or "Create New Virtual Machine."
 - Set the VM name, type (Mac OS X), and version (e.g., macOS Monterey).
3. **Allocate Resources:**
 - Assign memory (minimum 4 GB recommended).
 - Create a virtual hard disk (50 GB or more recommended).
4. **Modify VM Settings:**
 - Go to VM settings and enable EFI mode.
 - Add the macOS installation file as the bootable disk.
5. **Install macOS:**
 - Start the VM.
 - Follow the installation prompts to install macOS.
 - Configure system preferences and user accounts.

Exercise:

SET A: Download and Install Virtualization Software

1. Follow the steps outlined in the theory section.
2. Take a screenshot of the installed software.

SET B : Set Up and Configure a Linux VM

1. Download the Ubuntu ISO.
2. Create a new VM and configure resources.
3. Install Ubuntu and take a screenshot of the desktop.

SET C: Set Up and Configure a Windows VM

1. Download the Windows ISO.
2. Create a new VM and configure resources.
3. Install Windows and take a screenshot of the desktop.

SET D : Set Up and Configure a macOS VM

1. Obtain a mac OS installation file.
2. Create a new VM and configure resources.
3. Install mac OS and take a screenshot of the desktop.

Assignment Evaluation

0: Not Done []

1: Incomplete []

2: Late Complete []

3: Need Improvement []

4: Complete []

5: Well Done []

Signature of Instructor

Date of submission ____/____/____

Assignment 2

Static Analysis of Malwares

Attempt Any Two

Theory:

Static Analysis Overview

Static analysis aims to:

- Understand the malware's code structure.
- Identify malicious functionality.
- Extract indicators of compromise (IOCs) such as file hashes, domain names, IP addresses, registry keys, or file paths.
- Detect obfuscation techniques and packing methods.

This process is safer than dynamic analysis because it doesn't involve running the malware, reducing the risk of system compromise.

Steps in Static Analysis

1. File Identification

- **File Type:** Use tools like `file` (Linux) or `PEiD` to identify the type of file (e.g., PE file, ELF file, script).
- **File Size and Metadata:** Check file properties, including timestamps and metadata, which may reveal when and where the malware was created.

2. Hashing

- Compute cryptographic hashes (e.g., MD5, SHA256) of the file. These serve as unique identifiers for the malware and can be checked against malware databases like VirusTotal.

3. Signature Scanning

- Tools like antivirus engines or YARA rules scan the file for known patterns or byte sequences associated with malware.

4. Strings Analysis

- **Tool:** Use `strings` (Linux) or dedicated tools.
- **Objective:** Extract human-readable text embedded in the binary.
- **Insights:**
 - Hardcoded URLs or IP addresses.
 - File paths, API function names, or error messages.
 - Command-and-control (C2) server details.

5. PE (Portable Executable) File Analysis

For Windows executables, analyzing the PE structure reveals key insights:

- **Headers:** Examine PE headers (e.g., DOS header, NT headers) for metadata.
- **Imports/Exports:** Analyze imported and exported functions to identify API calls.
 - Examples of malicious APIs:
 - `InternetConnect` (network communication)
 - `VirtualAlloc` (memory manipulation)
 - Exported functions in DLLs might suggest malware's capabilities.
- **Sections:**
 - Analyze sections like `.text`, `.data`, and `.rdata`.
 - Suspicious signs:
 - Packed or obfuscated code in the `.text` section.
 - Excessively large `.rsrc` (resource) sections.

6. Decompiled Code Analysis

- Use decompilers or disassemblers (e.g., IDA Pro, Ghidra, Radare2) to convert machine code into assembly or high-level pseudocode.
- Analyze:
 - Control flow.
 - Key functions and routines.
 - Malware logic and algorithms (e.g., encryption, encoding).

7. Obfuscation and Packing Detection

- Malwares often use obfuscation or packing to avoid detection.
- Signs:
 - Unreadable strings.
 - High entropy sections (indicative of compressed or encrypted data).
- Tools:
 - PEiD or Detect It Easy (DIE) for packing detection.
 - Unpacking tools (e.g., UPX) or manual unpacking using debugging tools like OllyDbg.

8. Embedded Resources

- Extract embedded files, images, or executables.
- Inspect resource sections (`.rsrc`) for malicious payloads.

9. Configuration Extraction

- Extract configurations or encoded data embedded within the malware.
- Reverse any encoding schemes (e.g., Base64, XOR).

10. Cross-Referencing with Threat Intelligence

- Use tools and platforms like Virus Total, Hybrid Analysis, or Cuckoo Sandbox to compare the findings with known malware samples.

Exercise:

SET A:

1. xxd

Purpose:

Hexadecimal dump tool. Converts binary data into a hex dump and vice versa.

Installation:

- **Debian/Ubuntu:**

```
bash
```

```
sudo apt update
```

```
sudo apt install xxd
```

- **RHEL/CentOS:**

```
bash
```

```
sudo yum install vim-common
```

```
(xxd is part of the vim-common package.)
```

- **Arch Linux:**

```
Bash
```

```
sudo pacman -S vim
```

Usage:

- Hex dump:

```
bash
```

```
xxd file_name
```

- Reverse hex dump:

```
bash
```

```
xxd -r hex_dump_file > original_file
```

2. file

Purpose:

Identifies the type of a file (e.g., PE, ELF, script).

Installation:

- **Debian/Ubuntu:**

```
bash
```

```
sudo apt update
```

```
sudo apt install file
```

- **RHEL/CentOS:**

```
bash
sudo yum install file
```

- **Arch Linux:**

```
bash
sudo pacman -S file
```

Usage:

- Identify the file type:

```
bash
file file_name
```

3. md5sum

Purpose:

Generates an MD5 hash of a file.

Installation:

Included in the **coreutils** package, which is pre-installed on most Linux systems.

- If missing, install **coreutils**:
 - **Debian/Ubuntu:**

```
bash

sudo apt update
sudo apt install coreutils
```

- **RHEL/CentOS:**

```
bash

sudo yum install coreutils
```

- **Arch Linux:**

```
bash

sudo pacman -S coreutils
```

Usage:

- Generate MD5 hash:

```
bash
```

```
md5sum file_name
```

4. sha256sum

Purpose:

Generates a SHA-256 hash of a file.

Installation:

Also part of the **coreutils** package.

Usage:

- Generate SHA-256 hash:

```
bash
sha256sum file_name
```

5. sha1sum

Purpose:

Generates a SHA-1 hash of a file.

Installation:

Part of the **coreutils** package.

Usage:

- Generate SHA-1 hash:

```
bash
sha1sum file_name
```

Verify Installation

After installing, confirm each tool works by checking its version or using a test command:

```
bash

file --version
md5sum --version
sha256sum --version
sha1sum --version
xxd --version
```

SET B:

Download sample malware and perform static analysis. Print following info for the malwares using tools: File type, Cryptographic hash Values.

Perform the following tasks

1. Setup your environment:

- Use a dedicated virtual machine (e.g., Virtual Box or VMware) with a snapshot for quick recovery.
- Install tools such as:
 - **Hashing tool:** md5sum, sha256sum, or tools like HashMyFiles.
 - **Strings extraction:** strings (Linux/Windows), or Sysinternals Strings.
 - **Malware Scanning:** VirusTotal API or Metadefender Client (ensure the file is not sensitive before uploading).
- Install a hex editor (e.g., HxD or wxHexEditor) and a PE analysis tool (e.g., PEview or PEiD).

2. Download a sample malware or benign suspicious binary:

Use publicly available **benign samples** from sites like [Malware Traffic Analysis](#) or a fake malware sample (e.g., EICAR test file) from [EICAR's website](#).

3. Perform Static Analysis:

- a. **File Type:** Determine the file type (e.g., .exe, .dll, .bin, etc.) using file utilities or hex analysis.
 - b. **Hash Values:** Generate cryptographic hashes (e.g., MD5, SHA-1, SHA-256).
 - c. **Online Scan:** Upload the file to a scanner (e.g., VirusTotal, Jotti) and note the results.
 - d. **Extract Strings:** Use a strings extraction tool to identify human-readable content and potential clues.
4. **Write a report** summarizing your findings.

Example Solution (using benign EICAR test file)

1. File Type Analysis

- Command: `file eicar.com`
- Output: `eicar.com: ASCII text, with CRLF line terminators.`

2. Cryptographic Hashes

- MD5: 44D88612FEA8A8F36DE82E1278ABB02F
- SHA-1: 3395856CE81F2B7382DEE72602F798B642F14140
- SHA-256: 275A021BBFB6481CA693D7FF8F6A1A362F15E50F9B9E9E33B5F1A2C3000F9324.

3. Malware Scan Results

- Submitted to VirusTotal: Detected as "EICAR-Test-File" by most antivirus engines.
- Metadefender: Similar results as a benign test file.

4. Strings Extraction

- Command: `strings eicar.com`
- Output:

```
ruby
Copy code
X5O!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*
```

Notes on Real Malware Analysis

If you're analyzing real malware:

- **NEVER execute the file** unless you're in a sandbox.
- Do not upload sensitive malware to public scanning services (use private services or offline tools).
- Always document findings step by step for reproducibility.

SET C:

Write Python Script for the following

1. Print File type of suspicious malware.
2. Print Cryptographic hash Values of suspicious malware.
3. Accept the hash value (MD5/SHA1/SHA256) as input and queries the Virus Total database.

Answers:

1. Print File type of suspicious malware

1. **Install the Required Library:** If you don't already have the `python-magic` library installed, you can install it using `pip`. Open your terminal or command prompt and run:

```
bash
pip install python-magic
```

2. **Write the Python Script:** Below is a simple script that takes the path to a file as input and prints its file type using the `magic` library.

```
python
import magic
import os

def get_file_type(file_path):
    # Check if the file exists
    if not os.path.isfile(file_path):
        print(f"Error: The file '{file_path}' does not exist.")
        return

    # Get the file type
    file_type = magic.from_file(file_path)

    print(f"File Type of '{file_path}': {file_type}")

if __name__ == "__main__":
    # Input the path of the suspicious malware file
    file_path = input("Enter the path of the suspicious malware file: ")
```



```
get_file_type(file_path)
```

How to Run the Script

1. **Open Python IDLE:**
 - o Launch Python IDLE from your applications or terminal.
2. **Create a New File:**
 - o Go to File > New File to open a new window.
3. **Copy and Paste the Script:**
 - o Copy the provided script and paste it into the new file.
4. **Save the Script:**
 - o Save the file with a .py extension (e.g., file_type_checker.py).
5. **Run the Script:**
 - o Press F5 or go to Run > Run Module.
 - o When prompted, enter the full path to the suspicious malware file you want to analyze.

2 . Print Cryptographic hash Values of suspicious malware

```
import hashlib

def calculate_hashes(file_path):
    # Initialize hash objects
    md5_hash = hashlib.md5()
    sha256_hash = hashlib.sha256()

    # Read the file in binary mode
    try:
        with open(file_path, 'rb') as file:
            # Read and update hash string value in blocks of 4K
            for byte_block in iter(lambda: file.read(4096), b''):
                md5_hash.update(byte_block)
                sha256_hash.update(byte_block)

            # Get the hexadecimal digest of the hashes
            md5_digest = md5_hash.hexdigest()
            sha256_digest = sha256_hash.hexdigest()

            print(f"MD5: {md5_digest}")
            print(f"SHA-256: {sha256_digest}")

    except FileNotFoundError:
        print(f"Error: The file '{file_path}' was not found.")
    except Exception as e:
        print(f"An error occurred: {e}")

if __name__ == "__main__":
    # Specify the path to your suspicious malware file here
    file_path = input("Enter the path to the suspicious malware file: ")
    calculate_hashes(file_path)
```

How to Use the Script

1. Open Python IDLE.
2. Copy and paste the script into a new file or directly into the shell.
3. Save the file if using the file editor (e.g., `hash_calculator.py`).
4. Run the script:
 - o If in the shell, just press Enter after pasting the code.
 - o If saved as a file, use Run -> Run Module (or press F5).
5. When prompted, enter the full path to the suspicious malware file (e.g., `C:\path\to\your\malware.exe`).

Output

The script will output the MD5 and SHA-256 hash values of the specified file. If the file is not found or an error occurs, it will print an appropriate error message.

3 . Accept the hash value (MD5/SHA1/SHA256) as input and queries the Virus Total database

Prerequisites

1. **Install Requests Library:** If you haven't already, you'll need the `requests` library to make HTTP requests. You can install it via `pip`:

```
bash
Copy code
pip install requests
```

2. **API Key:** Obtain your VirusTotal API key from [VirusTotal](#).

Python Script

Here's the Python script to query the VirusTotal database using the hash value:

```
python
import requests

def query_virustotal(hash_value, api_key):
    # Set up the headers and endpoint for VirusTotal API
    url = f"https://www.virustotal.com/api/v3/files/{hash_value}"
    headers = {
        "x-apikey": api_key
    }

    # Make the request to the VirusTotal API
    response = requests.get(url, headers=headers)

    # Check the response status
    if response.status_code == 200:
        data = response.json()
        # Extract relevant information
        attributes = data.get('data', {}).get('attributes', {})
```

```

        last_analysis = attributes.get('last_analysis_results', {})

        print(f"Hash: {hash_value}")
        print(f"Scan Date: {attributes.get('last_analysis_date')}")
        print(f"Positives: {attributes.get('last_analysis_stats',
        {}).get('malicious', 0)}")
        print(f"Total: {attributes.get('last_analysis_stats', {}).get('harmless',
        0) + attributes.get('last_analysis_stats', {}).get('malicious', 0)}")

        print("\nDetailed Results:")
        for vendor, result in last_analysis.items():
            print(f"{vendor}: {result['result']}")
        else:
            print(f"Error querying VirusTotal: {response.status_code},
            {response.text}")

if __name__ == "__main__":
    # Input your VirusTotal API key here
    API_KEY = "YOUR_VIRUSTOTAL_API_KEY" # Replace with your actual API key
    hash_value = input("Enter the hash value (MD5/SHA1/SHA256): ")
    query_virustotal(hash_value, API_KEY)

```

How to Run the Script

1. **Open Python IDLE:** Open the Python IDLE environment.
2. **Copy and Paste the Script:** Paste the script into a new file or the interactive shell.
3. **Replace the API Key:** Replace YOUR_VIRUSTOTAL_API_KEY in the script with your actual API key.
4. **Run the Script:** Execute the script, and when prompted, enter the hash value you want to query.

Notes

- Ensure you have an active internet connection, as the script makes requests to the VirusTotal API.
- Free API accounts have usage limits, so check VirusTotal's terms and conditions for details.
- Make sure to handle the API key securely and not expose it in public repositories.

Assignment Evaluation

0: Not Done []

1: Incomplete []

2: Late Complete []

3: Need Improvement []

4: Complete []

5: Well Done []

Signature of Instructor

Date of submission ____/____/____

Assignment No 3

Dynamic Analysis of Malwares

Attempt Any Two

Goals:

- 1) Setup the lab/machine for dynamic analysis of malwares.
- 2) Install Process hacker and do analysis of sample malwares.
- 3) Install Process Monitor and do analysis of sample malwares.
- 4) Use noriben tool and do dynamic analysis of sample malwares.
- 5) Analyzing the DLL Using rundll32.exe

Theory:

1. Overview of Dynamic Analysis

Definition: Dynamic analysis is the process of executing malware in a controlled environment (like a sandbox) to monitor its behavior, system interactions, and the changes it makes to files, processes, and registry entries.

Objectives:

- Identify malicious actions performed by the malware.
- Observe communication with external servers (e.g., command and control).
- Understand how the malware propagates or persists on the system.
- Analyze the impact on system performance and security.

2. Setting Up a Dynamic Analysis Environment

A. Virtual Machines (VMs):

- Use virtualization platforms like VirtualBox or VMware to create isolated environments for testing.
- Configure VMs to have no internet access or limited connectivity to prevent malware from communicating with its command and control servers during testing.

B. Sandbox Environments:

- Use specialized sandbox solutions such as Cuckoo Sandbox, Any.run, or Hybrid Analysis, which automate the execution and monitoring of malware.
- Sandboxes provide controlled environments and can simulate various operating systems and user interactions.

C. Monitoring Tools:

- **Process Monitors:** Tools like Process Monitor (Sysinternals) can track file system, registry, and process activity.

- **Network Monitors:** Wireshark or TCPView can be used to capture and analyze network traffic generated by the malware.
- **System Resource Monitors:** Tools like Process Explorer or Task Manager help observe CPU, memory, and disk usage.

3. Dynamic Analysis Techniques

A. Behavioral Analysis:

- This technique focuses on observing how the malware behaves during execution.
- Monitor the creation of new files, registry modifications, and system calls.

B. Network Analysis:

- Analyze outbound connections to identify command and control (C2) communications.
- Capture network traffic to see if the malware communicates with external servers.

C. API Call Monitoring:

- Monitor the API calls made by the malware to understand its functionality and how it interacts with the operating system.
- Tools like API Monitor or Process Monitor can be useful.

D. Memory Analysis:

- Analyze the memory footprint of the malware to find hidden payloads or artifacts.
- Tools like Volatility or Rekall can help extract information from memory dumps.

4. Common Behavior Indicators

When performing dynamic analysis, look for the following behavior indicators:

- **File Modifications:** Creation, deletion, or modification of files.
- **Registry Changes:** Addition or modification of registry keys related to startup programs or system configuration.
- **Network Activity:** Unexpected outbound connections to unknown IP addresses or domains.
- **Process Creation:** Launching of new processes or termination of legitimate processes.
- **Persistence Mechanisms:** Techniques used to ensure the malware runs on system reboot (e.g., registry keys, startup folders).

5. Analyzing Results

Once the dynamic analysis is complete, analyze the gathered data:

- **Review Logs:** Analyze logs generated by monitoring tools to correlate actions and behaviors observed.
- **Identify Malicious Patterns:** Look for known malicious indicators (YARA rules, IOCs).
- **Document Findings:** Record all observed behaviors, including file names, registry keys, network traffic, and other relevant data.

6. Tools for Dynamic Analysis

- **Cuckoo Sandbox:** An open-source automated malware analysis system that allows you to execute and analyze malware in a controlled environment.
- **Any.run:** An interactive online malware analysis platform that provides real-time insights into malware behavior.
- **Wireshark:** A network protocol analyzer that captures and inspects network traffic.
- **Process Monitor:** A Sysinternals tool that logs file system, registry, and process/thread activity.
- **Sandboxie:** A tool that allows you to run programs in an isolated environment to prevent them from affecting the system.

7. Best Practices

- **Isolate the Environment:** Always conduct dynamic analysis in a contained environment to prevent the spread of malware.
- **Use Snapshots:** Use VM snapshots to revert to a clean state before each analysis.
- **Monitor Closely:** Use multiple monitoring tools to capture comprehensive data during execution.
- **Follow Legal Guidelines:** Ensure that you are compliant with local laws and regulations regarding malware analysis.

Exercise:

SET A:

Task 1: Setting Up the Lab Environment

1. **Virtual Machine (VM):**
 - Use a virtualization platform like VMware or VirtualBox to create an isolated environment.
 - Install an OS that you want to analyze the malware on (e.g., Windows 10).
2. **Network Configuration:**
 - Isolate the VM from the internet to prevent the malware from communicating with external servers. You can achieve this by using a Host-only network or an Internal network setting.
3. **Snapshots:**
 - Take a snapshot of the VM before any analysis begins. This allows you to revert to a clean state after running the malware.

Task 2: Installing Process Hacker

1. **Download Process Hacker:**
 - Go to the official website and download the latest version of Process Hacker.
 - Process Hacker
 - <https://processhacker.sourceforge.io/> (Download From here)
2. **Install Process Hacker:**
 - Run the installer and follow the prompts to complete the installation.
3. **Analyze Malware:**
 - After installation, launch Process Hacker.
 - Start the malware sample in the VM.
 - Use Process Hacker to observe the following:
 - Running processes: Look for suspicious processes.
 - Services: Check for any new services started by the malware.
 - Network connections: Monitor any unexpected connections made by the malware.

Task 3: Install Process Monitor and Analyze Sample Malware

1. **Install Process Monitor (ProcMon):**
 - Download from the official Sysinternals website: <https://learn.microsoft.com/en-us/sysinternals/downloads/procmon>.
 - Extract the zip file and run `Procmon.exe` as Administrator.
2. **Set Filters in ProcMon:**
 - Open ProcMon and stop live capture by clicking the magnifying glass icon (Ctrl+E).
 - Go to `Filter > Filter....`

- Set the filter for your sample malware executable (e.g., Process Name is `sample_malware.exe`).
- Click Add, then OK.
- 3. **Run the Sample Malware:**
 - Ensure you're using a virtual machine for safety.
 - Execute the sample malware file.
- 4. **Monitor Malware Activity:**
 - Observe registry, file system, and network activities in ProcMon.
 - Note any suspicious file creation, registry changes, or outgoing network connections.
- 5. **Save the ProcMon Log:**
 - Go to File > Save, select Native Process Monitor Format (PML), and save the log.

Task 5: Use Noriben for Dynamic Analysis

1. **Download and Setup Noriben:**
 - Download Noriben from <https://github.com/Rurik/Noriben>.
 - Ensure Python is installed and run `Noriben.py`.
2. **Start Noriben Monitoring:**
 - Open Command Prompt and navigate to the Noriben directory.
 - Run `python Noriben.py`.
3. **Run the Malware Sample:**
 - Follow Noriben's instructions to execute the malware.
4. **Analyze the Output Report:**
 - Noriben generates a detailed report (`Noriben_Report.txt` or similar).
 - Review the log for malicious activities, such as file manipulations, registry edits, or network communication.

Task 6 : Analyzing DLLs Using rundll32.exe

1. **Understand the DLL:**
 - Identify the DLL you wish to analyze (e.g., `malicious.dll`).
2. **Run the DLL with rundll32.exe:**
 - Open Command Prompt as Administrator.
 - Use the command:

```
bash
Copy code
rundll32.exe malicious.dll,ExportedFunction
```

 - Replace `ExportedFunction` with the DLL's entry point (use tools like Dependency Walker to find this).
3. **Monitor Activity:**
 - Use Process Monitor or Noriben to observe the DLL's behavior.
 - Check for file creation, registry edits, and network connections.
4. **Document Observations:**
 - Note the impact of running the DLL on the system.

Sample Observations and Expected Answers

1. ProcMon Output:

- Suspicious registry keys modified:
 - `HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\Run.`
- Files dropped in system directories, e.g., `C:\Windows\Temp.`
- Attempts to connect to suspicious IPs.

2. Noriben Report:

- The report highlights activities such as process injection, file creation, and network connections.
- Malware's persistence mechanisms (e.g., registry autostart).

3. DLL Analysis:

- Running `rundll32.exe` shows network activity to an external server.
- DLL injects code into another process (e.g., `explorer.exe`).

Assignment Evaluation

0: Not Done []

1: Incomplete []

2: Late Complete []

3: Need Improvement []

4: Complete []

5: Well Done []

Signature of Instructor

Date of submission ____/____/____

Assignment No 4

Disassembly using IDA

Attempt Any One

Task 01: Installation of IDA and IDA Python

Theory:

IDA (Interactive Disassembler) is a powerful tool for reverse engineering and analyzing binary files. **IDAPython** extends IDA's functionality by enabling Python scripting for automation.

Steps:

1. Download IDA:

- Visit the official Hex-Rays website and download IDA Free (<https://hex-rays.com/ida-free/>).
- Install IDA Free on your system.

2. Install IDAPython:

- IDAPython is bundled with most IDA versions. To confirm:
 - Go to `File > Script File...` in IDA.
 - Select any `.py` file to check for script execution capability.
- If not bundled:
 - Download IDAPython from <https://github.com/idapython/src>.
 - Follow the installation instructions provided on the GitHub page.

Verification:

- Open IDA and load a binary file.
- Run a simple Python script:

```
python
Copy code
print("Hello from IDAPython!")
```

Task 2: Static Code Analysis of Sample Malware

Theory:

Static code analysis involves examining a binary without executing it. IDA's disassembler decompiles binaries into assembly code for analysis.

Steps:

1. Load the Malware Sample:

- Open IDA and load the malware executable (`sample_malware.exe`).
- Let IDA analyze the binary.

2. Identify Key Sections:

- Navigate to the `.text` section for code and `.rdata` for strings.
- Check the **Imports** tab for API calls (e.g., `CreateFile`, `RegSetValueEx`, `WinExec`).
- 3. **String Analysis:**
 - Use the `View > Open Subviews > Strings` menu to find hardcoded strings like file paths or URLs.
- 4. **Function Analysis:**
 - Double-click suspicious functions (e.g., ones that reference imports).
 - Analyze the assembly instructions and flow graphs (Ctrl+G for the graph view).

Expected Output:

- Identify suspicious strings such as:
 - `http://malicious-site.com`.
 - Dropped file paths.
- Find key API calls like:
 - `WinExec` for executing commands.
 - `WriteFile` for creating files.

Task 03: Modify Malicious DLL to Execute notepad.exe

Theory:

DLL files contain executable code and can be modified to change behavior. Using `rundll32.exe`, malicious DLLs can be loaded and executed.

Steps:

1. **Open the Malicious DLL in IDA:**
 - Load the DLL file in IDA.
2. **Locate the Entry Point:**
 - Use the `Functions` window to find the exported functions.
 - Identify the function invoked by `rundll32.exe`.
3. **Modify the Behavior:**
 - Replace the original payload with code to launch `notepad.exe`:
4. **Execute the DLL:**
 - Save the modified DLL using a hex editor if IDA doesn't allow direct edits.
 - Run the DLL via `rundll32.exe` in Command Prompt:

```
asm
Copy code
push offset notepad_path ; Path to notepad.exe
call WinExec
ret
```

```
bash
Copy code
rundll32.exe malicious.dll,ExportedFunction
```

Expected Output:

- The modified DLL launches `notepad.exe` instead of executing malicious payloads.

Task 4: Static Code Analysis Using IDAPython

Theory:

IDAPython automates analysis by scripting repetitive tasks.

Steps:

1. Load the Malware Sample in IDA:

- Open the binary in IDA.

2. Write an IDAPython Script:

- Example script to list functions and imports:

```
python
Copy code
for func_ea in idutils.Functions():
    print(f"Function: {idc.get_func_name(func_ea)}")

for ea in idutils.Names():
    print(f"Name: {ea}, Address: {idc.get_name(ea)}")
```

3. Run the Script:

- Save the script as `analysis.py` and run it via `File > Script File`.

4. Analyze Results:

- The script outputs function names and imported API calls, enabling rapid identification of suspicious behavior.

Expected Output:

- List of functions in the malware.
- Imported API calls like `RegCreateKey`, `CreateFile`, or `WinExec`.

Assignment Evaluation

0: Not Done [☐]

1: Incomplete [☐]

2: Late Complete [☐]

3: Need Improvement [☐]

4: Complete [☐]

5: Well Done [☐]

Signature of Instructor

Date of submission __/__/__

Assignment No 5

Disassembly using IDA

Solve Any Two Theory

1. x64dbg

A popular open-source debugger for Windows applications, suitable for analyzing malicious executables and DLLs.

- **Features:**
 - Step-through debugging.
 - Code analysis and breakpoint setting.
 - Visualization of stack, memory, and registers.

2. dnSpy

A decompiler and debugger for .NET applications.

- **Features:**
 - View and edit .NET assemblies.
 - Debugging with IL (Intermediate Language) code visibility.
 - Decompile managed code into C#.

3. IDA (Interactive Disassembler)

A static analysis tool with debugging capabilities for analyzing complex binaries and DLLs.

- **Features:**
 - Graphical representation of program flow.
 - Code reverse engineering.
 - Dynamic analysis for runtime insights.

Exercise

Task 1: Install x64dbg and dnSpy

1. **x64dbg Installation:**
 - Download from <https://x64dbg.com>.
 - Extract the archive and run `x64dbg.exe` or `x32dbg.exe` (for 32-bit binaries).
2. **dnSpy Installation:**
 - Download from <https://github.com/dnSpy/dnSpy>.
 - Extract and run `dnSpy.exe`.

Task 2: Debugging a Sample Binary Using x64dbg

1. **Load the Malicious Binary:**
 - Open x64dbg.
 - Drag and drop the binary (e.g., `malicious.exe`) into x64dbg.
2. **Set Breakpoints:**
 - Locate the `EntryPoint` in the Disassembler view.
 - Right-click and set a breakpoint (F2).
3. **Analyze Execution Flow:**
 - Start debugging (F9).
 - Use step-into (F7) or step-over (F8) to analyze the code.
 - Monitor changes in registers, stack, and memory.

Task 3: Debugging a Sample DLL Using x64dbg

1. **Load the DLL:**
 - Open x64dbg.
 - Attach to a host application using `File > Attach`.
2. **Trigger the DLL Functions:**
 - Use `rundll32.exe` or another host program to load the DLL.
3. **Analyze Exported Functions:**
 - In the Symbols window, view the DLL's exported functions.
 - Set breakpoints on key functions.
4. **Trace Malicious Behavior:**
 - Monitor API calls and interactions with system resources.

Task 4: Debugging a Malicious Executable Using x64dbg

1. **Load the Executable:**
 - Open the malware in x64dbg.
2. **Monitor Suspicious Behavior:**
 - Look for API calls like `CreateFile`, `WriteFile`, `RegSetValue`, or `Send`.
 - Use the stack view to identify function parameters.
3. **Analyze Obfuscation Techniques:**
 - Identify and bypass packers or obfuscated code by locating jump instructions or decoding routines.

Task 5: Debugging a DLL Using IDA

1. **Load the DLL in IDA:**
 - Open IDA and load the DLL.
 - Select `Manual Load` to avoid automatic execution of code.

2. Static Analysis:

- Explore the functions and cross-references in the decompiled view.
- Use graph view to trace function calls and control flow.

3. Dynamic Debugging:

- Attach a debugger to the process running the DLL.
 - Set breakpoints on critical API calls.
-

Task 6: Debugging a .NET Application Using dnSpy

1. Load the .NET Binary:

- Open the .NET executable in dnSpy.

2. Decompile Code:

- View and analyze the decompiled C# code.

3. Debug the Application:

- Set breakpoints on suspicious methods.
- Run the application in debug mode (F5).

4. Trace Malicious Functions:

- Identify key behaviors like obfuscated strings or suspicious method calls.

Assignment Evaluation

0: Not Done []

1: Incomplete []

2: Late Complete []

3: Need Improvement []

4: Complete []

5: Well Done []

Signature of Instructor

Date of submission ____/____/____

Assignment No 6

Introduction to Malware Entry Points and Safeguarding It.

Theory

Malware Entry Points:

Malware can infiltrate systems through various entry points, including:

- **Email Attachments:** Malicious attachments or links in phishing emails.
- **Web Downloads:** Infected files downloaded from compromised or malicious websites.
- **USB Drives:** Malware transferred through infected USB devices.
- **Network Vulnerabilities:** Exploits targeting weaknesses in network security.
- **Social Engineering:** Techniques that trick users into executing malware.

Safeguarding Against Malware:

- **User Education:** Train users to recognize phishing and suspicious emails.
- **Antivirus Software:** Use updated antivirus and anti-malware solutions.
- **Firewalls:** Implement firewalls to block unauthorized access.
- **Regular Updates:** Keep software and operating systems updated to patch vulnerabilities.
- **Email Filtering:** Use email filters to identify and block spam or phishing emails.

Practical Assignment

1. Identify Entry Points:

- Conduct a risk assessment of your organization or home network.
- List potential entry points for malware.
- Suggest mitigation strategies for each entry point.

2. Create an Awareness Program:

- Develop a short presentation (5-10 slides) educating users about malware entry points and prevention techniques.

2. Emails, SPF, Spam Email Detection

Theory

Email Security:

- **Spam Emails:** Unsolicited messages often containing advertisements, scams, or malware.
- **SPF (Sender Policy Framework):** A method used to prevent spoofing by specifying which mail servers are authorized to send emails on behalf of a domain.

Spam Detection Techniques:

- **Content Filtering:** Analyzing the content of emails for spam indicators.

- **Header Analysis:** Checking email headers for inconsistencies or spoofing.
- **Bayesian Filtering:** Using statistical techniques to classify emails as spam or legitimate.

Practical Assignment

1. **SPF Record Implementation:**
 - Use an SPF record testing tool (like MxToolbox) to check the SPF records of a domain.
 - Suggest modifications to the SPF record to improve email authentication.
 2. **Spam Email Detection:**
 - Collect a set of emails (legitimate and spam) and analyze their headers.
 - Create a report identifying common spam characteristics and how they differ from legitimate emails.
-

3. Phishing

Theory

Phishing:

A fraudulent attempt to obtain sensitive information (like usernames, passwords, or credit card details) by masquerading as a trustworthy entity in electronic communications.

Common Types of Phishing:

- **Email Phishing:** Fake emails designed to trick users into providing sensitive information.
- **Spear Phishing:** Targeted phishing attacks aimed at specific individuals or organizations.
- **Whaling:** Phishing attacks targeting high-profile individuals, such as executives.

Preventive Measures:

- **User Training:** Educate users about recognizing phishing attempts.
- **Email Filtering:** Use advanced spam filters to detect phishing emails.
- **Two-Factor Authentication (2FA):** Implement 2FA to add an additional layer of security.

Practical Assignment

1. **Phishing Simulation:**
 - Create a simulated phishing email and present it to a small group (with prior consent).
 - Track how many individuals click the link or provide information.
 - Analyze the results and discuss how to improve awareness.
 2. **Phishing Report Creation:**
 - Research recent phishing trends and create a report summarizing the findings.
 - Include examples of phishing emails and effective responses.
-

4. USB Security

Theory

USB Security Risks:

USB devices can be exploited to deliver malware, steal data, or compromise security. Common risks include:

- **Infected USB Drives:** Malware can be spread through removable drives.
- **BadUSB Attacks:** USB devices reprogrammed to execute malicious commands.
- **Data Leakage:** Unauthorized data transfer from secure systems.

Preventive Measures:

- **Endpoint Security:** Implement security measures on USB ports to control device access.
- **User Policies:** Create policies for the use of USB devices in the organization.
- **Data Encryption:** Encrypt sensitive data to protect against unauthorized access.

Practical Assignment

1. USB Security Assessment:

- Conduct an audit of USB device usage within your organization.
- Identify unauthorized devices and assess their potential risks.
- Create a report with recommendations for improving USB security.

2. Data Encryption Practice:

- Choose a USB drive and encrypt it using tools like BitLocker (Windows) or VeraCrypt (cross-platform).
- Document the steps taken to encrypt the USB drive and how to securely manage the password.

Assignment Evaluation

0: Not Done []

1: Incomplete []

2: Late Complete []

3: Need Improvement []

4: Complete []

5: Well Done []

Signature of Instructor

Date of submission ____/____/____

Assignment No 7

Toolkit for Malware Analysis

What is REMnux?

REMnux is a Linux distribution specifically designed for malware analysis and reverse engineering. It provides a comprehensive suite of open-source tools that enable security professionals and researchers to analyze malicious software and understand its behavior.

Key Features of REMnux:

1. **Pre-installed Tools:**

REMnux comes with a variety of tools pre-installed for different types of malware analysis, including:

- **Static Analysis Tools:** Tools for examining malware without executing it, such as strings, binwalk, and apktool.
- **Dynamic Analysis Tools:** Tools that allow users to run and observe malware in a controlled environment, including Cuckoo Sandbox, x64dbg, and Ghidra.
- **Network Analysis Tools:** Tools for monitoring network traffic generated by malware, such as Wireshark and tcpdump.

2. **Malware Forensics:**

REMnux is equipped with tools that help in digital forensics, allowing users to investigate the artifacts left by malware on compromised systems.

3. **Documentation and Community Support:**

The distribution is backed by extensive documentation and an active community, providing users with guidance on using the tools and performing malware analysis effectively.

4. **Lightweight and Customizable:**

REMnux is designed to be lightweight and can be run as a live environment or installed on virtual machines, making it versatile for various analysis scenarios.

5. **Regular Updates:**

The tools and features within REMnux are regularly updated to keep pace with evolving malware techniques and tactics.

Practical Steps

Part 1: Set Up REMnux

1. **Download and Install REMnux:**

- Download the latest version of REMnux from the official website: [REMnux Download](#).
- Create a new virtual machine using VirtualBox, VMware, or another hypervisor.
- Import the REMnux OVA file or install it using the provided instructions.
- Allocate at least 2GB of RAM and 20GB of disk space for the VM.

2. **Configure the VM:**

- Enable networking for internet access to install updates and tools.
- Ensure that the VM's settings allow for shared clipboard and drag-and-drop features for ease of use.

Part 2: Explore REMnux Tools

1. **Access the Terminal:**
 - Open the terminal in REMnux.
 - Use the command `remnux` to get an overview of the tools available.
2. **Familiarize with Key Tools:**
 - List and briefly describe tools such as:
 - **Cuckoo Sandbox:** Automated malware analysis.
 - **Ghidra:** Reverse engineering tool.
 - **Volatility:** Memory analysis tool.
 - **PEStudio:** Windows executable analysis.
 - Use commands like `apt list --installed` to see the installed tools.

Part 3: Static Analysis of a Malware Sample

1. **Download a Malware Sample:**
 - Obtain a safe malware sample for analysis (e.g., a known sample from [VirusShare](#) or [MalwareBazaar](#)).
 - Ensure the sample is safe to analyze and ideally within a controlled environment.
2. **Analyze the Malware:**
 - Use the following tools for static analysis:
 - **strings:** Extract human-readable strings from the binary.

```
bash
Copy code
strings malware_sample.exe > strings_output.txt
```
 - **file:** Determine the type of the malware.

```
bash
Copy code
file malware_sample.exe
```
 - **pefile:** Analyze Portable Executable (PE) files.

```
bash
Copy code
python3 -m pefile malware_sample.exe
```
 - Document your findings, including any suspicious strings or indicators of compromise (IOCs).

Part 4: Dynamic Analysis of a Malware Sample

1. **Set Up a Controlled Environment:**
 - Use **Cuckoo Sandbox** or set up a simple environment to run the malware safely.
 - If using Cuckoo, follow the installation guide to set it up for analysis.
2. **Run the Malware Sample:**
 - Submit the malware sample to Cuckoo Sandbox or run it in a controlled environment.
 - Monitor the system's behavior using **Wireshark** to capture network activity and **Sysinternals Suite** to monitor system changes.
3. **Observe and Record Findings:**

- Record network connections made by the malware.
- Note any files created, modified, or deleted during execution.
- Identify any registry changes if applicable.

Part 5: Document Findings

1. Create an Analysis Report:

- Summarize the static and dynamic analysis results in a report.
- Include:
 - Sample details (filename, hash, etc.).
 - Static analysis results (suspicious strings, file type).
 - Dynamic analysis results (behavior observed, network activity).
 - Recommendations for detection or remediation.

2. Presentation:

- Prepare a short presentation (5-10 slides) highlighting your findings and key insights from the analysis.

Assignment Evaluation

0: Not Done []

1: Incomplete []

2: Late Complete []

3: Need Improvement []

4: Complete []

5: Well Done []

Signature of Instructor

Date of submission ____/____/____

Assignment No 8

Introduction to Network Analysis

Theory

1. Set Up Wireshark

- Install and configure Wireshark on your machine.
- Capture network traffic.

2. Analyze Captured Traffic

- Examine the captured packets for signs of malicious activity.
- Identify infected hosts and their communications.

3. Create a Report

- Summarize your findings and present recommendations.

Exercise

Part 1: Set Up Wireshark

1. Install Wireshark:

- Download Wireshark from the official website: [Wireshark Download](#).
- Follow the installation instructions for your operating system.

2. Configure Wireshark:

- Launch Wireshark and select the appropriate network interface to capture traffic (e.g., Wi-Fi, Ethernet).
- Set up capture filters if necessary to limit the amount of data (e.g., only capture HTTP traffic: `tcp port 80`).

3. Start Capturing Traffic:

- Begin capturing packets by clicking the "Start Capturing Packets" button.
- Allow Wireshark to capture traffic for at least 10-15 minutes to gather sufficient data.

Part 2: Analyze Captured Traffic

1. Stop the Capture:

- After collecting sufficient traffic, click the "Stop Capturing Packets" button.

2. Initial Review:

- Use the display filter to focus on potential signs of infection. Some useful filters include:
 - **HTTP Traffic:** `http`
 - **DNS Queries:** `dns`
 - **TCP Connections:** `tcp`
 - **Malicious Domains or IPs:** If you have specific domains or IP addresses to investigate, you can use filters like `ip.addr == <IP_Address>` or `http.host == <domain>`.

3. Look for Suspicious Patterns:

- Review the captured traffic for unusual behavior, such as:
 - Frequent connections to known malicious domains.
 - Unusual outbound traffic from internal IPs.
 - Large amounts of data being sent to unknown external IPs.
 - DNS requests for suspicious domains or domains with misspellings.

4. Identify Infected Hosts:

- Look for specific IP addresses or hostnames that show abnormal behavior.
- Analyze the source and destination of suspicious traffic to trace back to the infected hosts.

Part 3: Create a Report

1. Document Your Findings:

- Create a report that includes:
 - An overview of the traffic captured (total packets, protocols used, etc.).
 - A summary of any suspicious activity identified (e.g., specific IPs or domains).
 - Details of any infected hosts, including their IP addresses and any observed malicious behavior.

2. Recommendations:

- Provide recommendations based on your analysis, which may include:
 - Suggestions for further investigation (e.g., deeper forensic analysis of identified hosts).
 - Network segmentation to isolate infected machines.
 - Implementing additional security measures like intrusion detection systems (IDS).

3. Presentation:

- Prepare a short presentation (5-10 slides) to share your findings with your peers or colleagues. Include key visualizations from Wireshark, such as packet flows, protocol distribution, or graphs illustrating suspicious activity.

Assignment Evaluation

0: Not Done []

1: Incomplete []

2: Late Complete []

3: Need Improvement []

4: Complete []

5: Well Done []

Signature of Instructor

Date of submission ____/____/____

Assignment No 9

Research Paper Activity

Write Down one research Paper from the following

- Writing review paper on Static analysis of Malwares.
- Writing review paper on Dynamic analysis of Malwares.
- Writing review paper on Hybrid analysis of Malwares.

Assignment Evaluation

0: Not Done []

1: Incomplete []

2: Late Complete []

3: Need Improvement []

4: Complete []

5: Well Done []

Signature of Instructor

Date of submission ____/____/____